

Descripción del protocolo cliente-servidor

Monopoly Distribuido con Estructuras Lineales

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Algoritmos y Estructuras de Datos 1 · Semestre 2 2026

Integrantes: Coto Solano Fabricio De Jesus

Esquivel Alvarez Christian David

Garcia Castro Isaac Josue

Rodriguez Cordero Ulfran Ricardo

1 de octubre de 2026

1. Visión general

El servidor mantiene el estado oficial de la partida y los clientes solo le piden acciones por TCP.

- **Servidor (banco):** aplicación de consola en C# que corre en la computadora del organizador. Escucha en el puerto TCP 5000 y atiende a cada cliente en un hilo propio.
- **Clientes:** una ventana de la interfaz en Godot por jugador, hasta 4. Se conectan a la IP del servidor y dibujan el estado que este les devuelve.
- **Hardware:** la Raspberry Pi Pico (dado electrónico) y el lector RFID se conectan por USB al servidor, no a los clientes.

El cliente no puede modificar saldo, posición, propiedades, turno, dados ni transacciones. Solo envía el nombre de una acción; el servidor la valida y, si procede, cambia el estado.

2. Transporte y formato

Cada mensaje es un objeto JSON en una sola línea, enviado por un socket TCP.

- **Transporte:** sockets TCP, servidor en el puerto 5000.
- **Codificación:** UTF-8.
- **Delimitador:** cada mensaje termina con un salto de línea (`\n`). El receptor lee línea por línea, así que no importa si TCP junta o corta paquetes.
- **Un solo tipo de mensaje:** la clase `Mensaje` sirve para las solicitudes del cliente, las respuestas del servidor y los avisos a los demás clientes.
- **Nombres de campos:** en PascalCase (`Accion`, `IdJugador`, `Exito`, `Descripcion`, `Datos`).
- **Fin de conexión:** si el receptor lee una línea nula, el otro lado cerró el socket y se libera la conexión.

El primer mensaje de un cliente es siempre `CONECTAR`. A partir de ahí el cliente mantiene un hilo que solo escucha al servidor, para recibir respuestas y avisos sin bloquear la interfaz.

3. Estructura del mensaje

Todo mensaje tiene cinco campos, y solo **Accion** e **IdJugador** son obligatorios en una solicitud.

Campo	Tipo	Lo llena	Contenido
Accion	enum TipoAccion	cliente y servidor	Acción pedida. La respuesta repite la acción de la solicitud.
IdJugador	entero (1 a 4)	cliente	Jugador que envía el mensaje. La respuesta lo repite.
Exito	booleano, opcional	servidor	true si la acción se realizó, false si el servidor la rechazó. Va vacío en las solicitudes.
Descripcion	texto, opcional	servidor	Texto legible para el jugador, por ejemplo “Ya se lanzaron los dados en este turno.”
Datos	JSON, opcional	cliente o servidor	Contenido propio de cada acción: el estado del juego, el historial o el valor de un dado.

Tabla 1: Campos de la clase **Mensaje**.

Datos es un JSON libre. Así no hace falta una clase distinta por cada tipo de mensaje.

4. Acciones del protocolo

El cliente puede pedir ocho acciones. Tres son consultas y no exigen que sea su turno.

Acción	Valor	Turno	Qué hace el servidor	Avisa
CONECTAR	0	No	Registra al jugador y responde con la bienvenida.	No
TIRAR_DADOS	1	Sí	Mueve al jugador por el tablero y procesa la casilla donde cae. Una sola vez por turno.	Sí
COMPRAR_PROPIEDAD	2	Sí	Compra la propiedad de la casilla actual si está disponible y alcanza el saldo. Descuenta el precio, asigna el dueño y registra la transacción.	Sí
NO_COMPRAR	3	Sí	El jugador declina la propiedad de la casilla actual.	No
TERMINAR_TURNO	4	Sí	Pasa el turno al siguiente jugador activo. Salta a los eliminados y a quien pierde el turno.	Sí
CONSULTAR_ESTADO	5	No	Devuelve en Datos el estado completo de la partida.	No
CONSULTAR_TRANSACCIONES	6	No	Devuelve en Datos el historial y regenera el archivo transacciones.txt .	No
PAGAR	7	Sí	Cubre un pago obligatorio del jugador.	Sí

Tabla 2: Acciones del protocolo. “Turno” indica si exige que sea el turno del jugador; “Avisa” si el servidor notifica a los demás clientes.

“Avisa” significa que, después de responder al cliente que pidió la acción, el servidor envía la misma respuesta a los otros clientes conectados.

5. Validaciones del servidor

El servidor revisa cada acción antes de tocar el estado, y si la rechaza responde con **Exito** en **false** y el motivo en **Descripcion**.

Regla	Respuesta cuando se incumple
Solo juega el jugador en turno	“No es el turno del jugador N. Es el turno de X.”
Una sola tirada de dados por turno	“Ya se lanzaron los dados en este turno.”
No se juega después del final	“La partida ya terminó.”
El jugador del turno debe estar activo	“No hay un jugador activo para este turno.”
Compra solo con saldo suficiente y sin dueño previo	El banco rechaza la compra y devuelve el motivo.
La acción debe existir en el protocolo	“Acción no reconocida o no soportada.”

Tabla 3: Validaciones y respuestas de rechazo.

Las consultas (CONECTAR, CONSULTAR_ESTADO y CONSULTAR_TRANSACCIONES) se aceptan fuera de turno porque no cambian el estado. Un jugador que no puede cubrir un pago obligatorio queda eliminado, y la partida termina con un solo jugador activo o al llegar al límite de turnos.

6. Respuestas y avisos a los clientes

Cada solicitud recibe una respuesta directa, y las acciones que cambian el estado se avisan además a los otros jugadores.

1. **Respuesta directa:** el servidor devuelve un **Mensaje** con la misma **Accion** e **IdJugador**, más **Exito** y **Descripcion**.
2. **Aviso a los demás:** tras un **TIRAR_DADOS**, **COMPRAR_PROPIEDAD**, **TERMINAR_TURNO** o **PAGAR** exitoso, el servidor reenvía la misma respuesta a todos los clientes conectados menos al que la pidió. Guarda las conexiones en un diccionario concurrente indexado por **IdJugador**.
3. **Sondeo del estado:** el dado de la Pico y el lector RFID actúan directo sobre el servidor, así que esos movimientos no generan un aviso. Por eso cada cliente pide **CONSULTAR_ESTADO** una vez por segundo y redibuja la pantalla con lo que recibe.

La interfaz nunca calcula nada: copia los saldos, las posiciones, los dueños y el turno que manda el servidor.

7. Estado del juego e historial

Las dos consultas devuelven su resultado en el campo **Datos**.

7.1. CONSULTAR_ESTADO

Campo	Tipo	Contenido
Turno	entero	Número del turno actual. Cada jugada de un jugador cuenta como un turno.
JugadorActualId, JugadorActualNombre	entero, texto	Jugador que tiene el turno.
JugadoresActivos	entero	Cuántos jugadores siguen en la partida.
JuegoTerminado	booleano	true cuando la partida ya acabó.
DadosLanzados	booleano	true si ya se tiró en este turno.
Jugadores	arreglo	Por jugador: Id, Nombre, Saldo, Patrimonio, Activo, EnCarcel y Posicion (el Id de su casilla).
Duenos	arreglo de enteros	Una entrada por casilla: el Id del jugador dueño, o -1 si no tiene dueño.
Ganador	texto o null	Nombre del jugador con mayor patrimonio, solo cuando la partida terminó.

Tabla 4: Campos de Datos en la respuesta de CONSULTAR_ESTADO.

Ejemplo abreviado de Datos:

```
{"Turno":1,"JugadorActualId":1,"JugadorActualNombre":"Christian","JugadoresActivos":4,
"JuegoTerminado":false,"DadosLanzados":false,
"Jugadores":[{"Id":1,"Nombre":"Christian","Saldo":1500,"Activo":true,"EnCarcel":false,"
  Posicion":0}, "..."],
"Duenos":[-1,-1,-1, "..."],"Ganador":null}
```

7.2. CONSULTAR_TRANSACCIONES

Datos es un arreglo con todas las transacciones, de la más antigua a la más reciente. Cada una trae Id, Fecha, Turno, Tipo, Origen, Destino, Monto y Descripcion. Cuando el origen o el destino es el banco, el campo dice “Banco”.

Cada consulta también regenera el archivo `transacciones.txt` en la computadora del servidor. Los tipos que genera una partida son: compra, alquiler, pago al banco, pago por evento, ganancia por evento y recompensa por pasar por Inicio.

8. Integración del hardware

La Pico y el lector RFID hablan con el servidor por USB, y el servidor trata sus eventos como si los hubiera enviado un cliente.

- **Dado (Raspberry Pi Pico):** manda por el puerto serie, a 115200 baudios, una línea `DADO:n` con la suma de los dos dados. `LocalSerialService` la lee, arma un mensaje `TIRAR_DADOS` para el jugador en turno con el valor en `Descripcion` y se lo pasa al procesador del servidor.
- **Valor del dado:** `TIRAR_DADOS` toma el valor de `Datos.valor` o de `Descripcion`. Si no llega ningún valor, el servidor lanza dados virtuales, y así el botón de la pantalla también funciona.
- **Validaciones:** una tirada de la Pico pasa por las mismas reglas que cualquier acción. Si el servidor la rechaza, imprime el motivo en su consola y no mueve a nadie.
- **Tras la tirada:** si la casilla no se puede comprar, el servidor termina el turno solo. Si se puede comprar, espera la compra con el botón de la pantalla o con la tarjeta RFID.

- **Lector RFID:** al acercar una tarjeta, el servidor lee su UID y ejecuta `COMPRAR_PROPIEDAD` para el jugador que tiene el turno. La tarjeta solo confirma la compra: el servidor no usa el UID para identificar al jugador. Si la compra se realiza, el turno termina.
- **Puertos:** al arrancar, el servidor busca la Pico (`cu.usbmodem*`) y el lector (`cu.usbserial*`) o los recibe como argumentos. Sin hardware, el servidor arranca igual.

9. Flujo de un turno

El dado y la tarjeta llegan al servidor por USB; los clientes consultan el estado y piden las demás acciones.

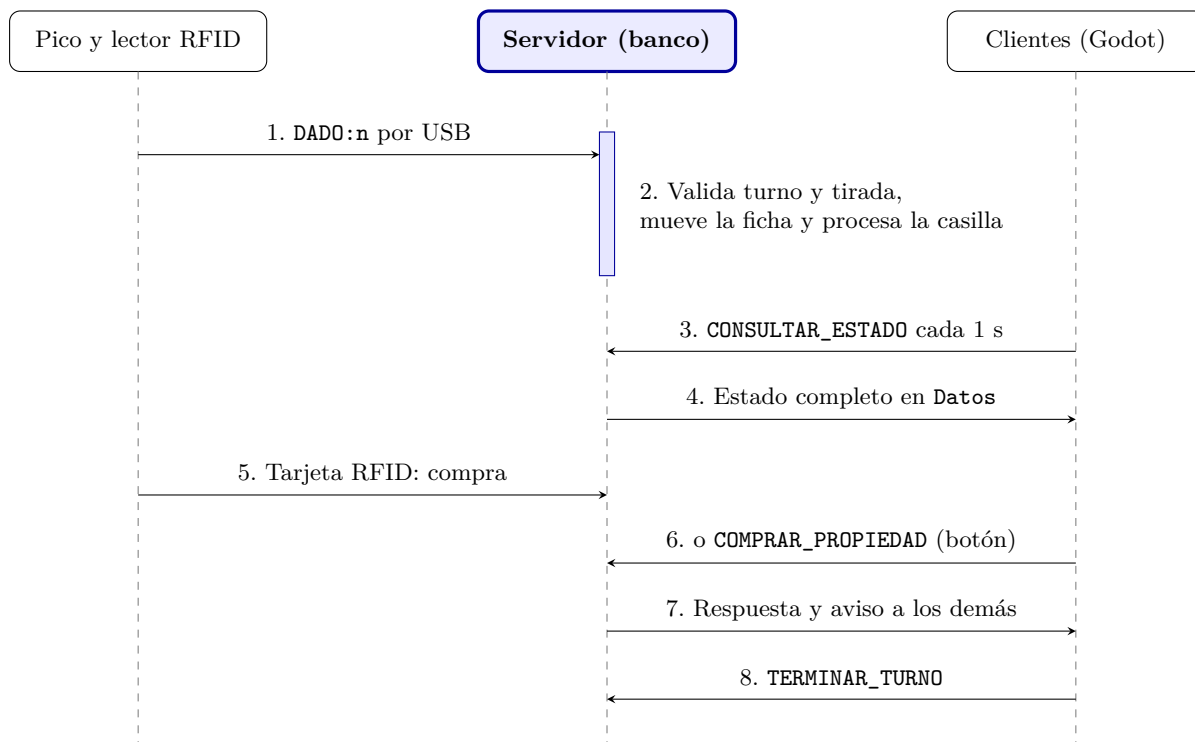


Figura 1: Flujo de un turno: el servidor decide cada paso y los clientes solo piden y dibujan.

9.1. Ejemplos de mensajes

Solicitud de un cliente que pide el estado (jugador 1):

```
{"Accion":5,"IdJugador":1,"Exito":null,"Descripcion":null,"Datos":null}
```

Respuesta del servidor cuando se intenta tirar dos veces en el mismo turno:

```
{"Accion":1,"IdJugador":1,"Exito":false,"Descripcion":"Ya se lanzaron los dados en este turno .","Datos":null}
```